

Translated

From: e-maxx.ru

Finding Bridges Online

We are given an undirected graph. A bridge is an edge whose removal makes the graph disconnected (or, more precisely, increases the number of connected components). Our task is to find all the bridges in the given graph.

Informally this task can be put as follows: we have to find all the "important" roads on the given road map, i.e. such roads that the removal of any of them will lead to some cities being unreachable from others.

There is already the article [Finding Bridges in \$O\(N+M\)\$](#) which solves this task with a [Depth First Search](#) traversal. This algorithm will be much more complicated, but it has one big advantage: the algorithm described in this article works online, which means that the input graph doesn't have to be known in advance. The edges are added once at a time, and after each addition the algorithm recounts all the bridges in the current graph. In other words the algorithm is designed to work efficiently on a dynamic, changing graph.

More rigorously the statement of the problem is as follows: Initially the graph is empty and consists of n vertices. Then we receive pairs of vertices (a, b) , which denote an edge added to the graph. After each received edge, i.e. after adding each edge, output the current number of bridges in the graph.

It is also possible to maintain a list of all bridges as well as explicitly support the 2-edge-connected components.

The algorithm described below works in $O(n \log n + m)$ time, where m is the number of edges. The algorithm is based on the data structure [Disjoint Set Union](#). However the implementation in this article takes $O(n \log n + m \log n)$ time, because it uses the simplified version of the DSU without Union by Rank.

Algorithm

First let's define a k -edge-connected component: it is a connected component that remains connected whenever you remove fewer than k edges.